

GTL Technical Guide

Introduction

GTL (Genesis Topology Language) is a Python object model for defining governed work graphs. A GTL system declares asset types, transitions between those assets, operators used on each transition, evaluation criteria, governance rules, and worker capabilities. Genesis is an engine that executes those definitions.

The authored surface is Python. The language can be documented cleanly today without inventing a separate text syntax.

GTL In One Page

GTL models work as typed transitions between typed assets.

An asset is a class of artifact such as `requirements`, `design`, `code`, or `interpretation_case`.

An edge is a transition between assets. It defines: - the source asset type - the target asset type - the operators used during the transition - the external context required by the transition - the governance rule, if the transition needs approval - the convergence basis for deciding when work on that edge is complete

Operators run in one of three regimes: - `F_D`: deterministic checks - `F_P`: probabilistic or agentic work - `F_H`: human judgment

Evaluators define the stopping conditions for a job. Jobs and workers turn the graph into an execution model.

The result is a single typed system that describes: - what can be produced - how it can be produced - what evidence is required - who or what can perform the work - how completion is assessed

What Problem GTL Solves

Most workflow definitions split across several surfaces: - topology in diagrams or YAML - acceptance criteria in prose - execution logic in code - approval rules in documentation or process

GTL keeps those concerns in one typed model.

A GTL definition gives an engine enough information to answer: - what assets exist in this domain - which transitions are legal - what context a transition depends on - which checks are deterministic - where judgment is required - which worker can safely execute which job

This makes the package reviewable before execution and auditable during execution.

The Authored Surface

The authored surface is Python.

Packages are written with imports from `gtl.core`:

```
from gtl.core import (
    Package, Asset, Edge, Operator, Rule, Context, Overlay,
    Evaluator, Job, Worker,
    F_D, F_P, F_H, consensus,
)
```

This guide assumes that surface. A separate `.gtl` syntax is not required to understand or use the current model.

Core Types

F_D, F_P, F_H

These classes define the evaluation regime.

`F_D` is for deterministic checks such as tests, schema validation, or threshold checks.

`F_P` is for agentic construction or evaluation where the result is useful but not fully deterministic.

`F_H` is for human approval or judgment.

consensus() and Rule

Approval rules are reusable objects.

```
hard_edge = Rule(
    name="hard_edge",
    approve=consensus(3, 4),
    dissent="required",
)
```

`consensus(n, m)` defines the approval ratio. Rule adds dissent handling and provisional status.

Context

Context binds external information into the package.

```
institutional_scope = Context(
    name="institutional_scope",
    locator="git://github.com/org/repo//ctx/scope.yml@abc123",
    digest="sha256:9c1d3f...",
)
```

`locator` is used to find the source material. `digest` is the stable binding for replay.

Operator

Operator declares the mechanism used on an edge.

```
pytest_op = Operator("pytest", F_D, "exec://python -m pytest tests/ -q")
human_gate = Operator("human_gate", F_H, "fh://single")
```

The current URI families are: `- agent://` `- exec://` `- check://` `- metric://` `- fh://`

Asset

Asset defines a typed artifact class.

```
requirements = Asset(
    name="requirements",
    id_format="REQ-{SEQ}",
    lineage=[intent],
)
```

```

    markov=["keys_testable", "intent_covered"],
)

```

lineage records the upstream asset types required for provenance.

markov lists the conditions that make an instance of the asset reusable without re-reading its full construction history.

Edge

Edge defines a transition between assets.

```

interpret = Edge(
    name="interpret",
    source=intent,
    target=requirements,
    using=[req_extract, human_gate],
    confirm="question",
    rule=hard_edge,
    context=[project_constraints],
)

```

The model supports three edge shapes: - unary transition: A -> B - product transition: [A, B] -> C - co-evolution: [A, B] with co_evolve=True

confirm defines the basis for completion: - question - markov - hypothesis

WorkingSurface

WorkingSurface is the structured output of job execution.

It contains: - emitted events - generated artifacts - consumed context

This gives the engine an evidence surface as well as a control surface.

Evaluator

Evaluator defines a convergence predicate.

```

tests_pass = Evaluator("tests_pass", F_D, "all unit tests pass")

```

Evaluators are the stopping conditions for work on a job.

Job

Job wraps an edge with its evaluators.

```

tdd_job = Job(edge=tdd_edge, evaluators=[tests_pass, coverage_ok])

```

This gives the edge an execution contract. A valid job has at least one evaluator.

Worker

Worker defines execution capability structurally.

```

claude = Worker(id="claude", can_execute=[tdd_job, design_job])

```

The worker's write territory is derived from the target asset types of the jobs it can execute.

This makes scheduling a type problem rather than a prose policy problem.

Overlay

Overlay defines lawful package change.

Two forms are supported: - restriction overlays through `restrict_to` - additive overlays through `add_assets`, `add_edges`, `add_operators`, `add_rules`, and `add_contexts`

Restriction overlays are the current profile mechanism.

PackageSnapshot

PackageSnapshot is the runtime binding of package law to execution events.

It records which version of the package governed a unit of work. It is a runtime artifact, not a hand-authored object.

Package

Package is the bounded domain model.

```
genesis_sdlc = Package(  
    name="genesis_sdlc",  
    assets=[intent, requirements, design, code, unit_tests],  
    edges=[interpret, design_from_requirements, tdd],  
    operators=[req_extract, pytest_op, human_gate],  
    rules=[hard_edge],  
    contexts=[project_constraints],  
)
```

Package validates that: - all operators used by edges are declared - `co_evolve=True` is used consistently - the graph is structurally well-formed at construction time

Execution Model

The package describes the domain graph. Jobs and workers describe how that graph is executed.

The execution model has four pieces: - Edge: transition structure - Evaluator: completion criteria - Job: executable transition plus evaluators - Worker: capability set over jobs

This separation matters.

The package says what the world looks like. Jobs and workers say how work moves through that world.

Minimal Example

This is the smallest useful GTL pattern.

```
from gtl.core import (  
    Package, Asset, Edge, Operator, Rule, Context,  
    Evaluator, Job, Worker,  
    F_D, F_P, F_H, consensus,  
)  
  
intent = Asset("intent", "INT-{SEQ}", markov=["stated"])  
requirements = Asset(  
    "requirements",  
    "REQ-{SEQ}",  
    lineage=[intent],  
    markov=["keys_testable", "intent_covered"],  
)  
  
constraints = Context(  
    "constraints",  
    locator="workspace://context/constraints.md",  
    digest="sha256:abcd",  
)  
  
extract = Operator("extract", F_P, "agent://requirements_extraction")  
approve = Operator("approve", F_H, "fh://single")
```

```

gate = Rule("gate", approve=consensus(1, 1), dissent="recorded")

interpret = Edge(
    name="interpret",
    source=intent,
    target=requirements,
    using=[extract, approve],
    confirm="question",
    rule=gate,
    context=[constraints],
)

approved = Evaluator("approved", F_H, "human approval recorded")

interpret_job = Job(edge=interpret, evaluators=[approved])
analyst = Worker(id="analyst", can_execute=[interpret_job])

package = Package(
    name="example_package",
    assets=[intent, requirements],
    edges=[interpret],
    operators=[extract, approve],
    rules=[gate],
    contexts=[constraints],
)

```

This example has: - two asset types - one transition - one context surface - one governance rule - one evaluator - one worker

That is enough to express a complete, governed transition.

Example Domains

GTL is designed to support materially different domains with one object model.

Software Delivery

This package family demonstrates: - staged work products such as requirements, design, code, and tests - co-evolution between code and tests - deterministic checks such as test execution and coverage - profile-style restriction overlays

Regulatory Obligations

This package family demonstrates: - source interpretation under human review - strong governance requirements - product arrows such as [normalized_obligation, activity_signature] -> applicability_binding - context surfaces tied to external authorities

Enterprise Architecture

This package family demonstrates: - architecture requirement capture - candidate design evaluation - governance gates over platform and technology choices - parallel exploration of alternatives

A language that handles all three with one object model is domain-general enough to support broader use.

Validation Rules

The object model enforces several rules at construction time.

1. Context.digest must use a sha256: binding.
2. Operator.category and Evaluator.category must be F_D, F_P, or F_H.
3. Edge.confirm must be question, markov, or hypothesis.
4. Job.evaluators must not be empty.
5. Worker.can_execute must not be empty.

6. Overlay must declare approval.
7. An overlay cannot mix restriction and additive forms.
8. Every operator used by an edge must be declared in the package.

These rules make package definitions rejectable before runtime.

Runtime Boundary

The boundary between authored model and runtime state matters for replay and audit.

Authored in Python: - package structure - operators - rules - contexts - evaluators - jobs - workers

Derived at runtime: - package snapshots - active traversal state - emitted work events - convergence state for specific artifact instances